

Supercharging Your $\text{\LaTeX}$ Workflow with Vim

Lance Remigio

Sacramento State University

November 3, 2025

What is the goal of this talk?

- ▶ The goal is to convert you into using vim for speeding up your $\text{\LaTeX}$ workflow.
- ▶ If not converted immediately, then at least you will explore more of vim and its many interesting aspects and decide for yourself if it is something worth pursuing.
- ▶ While we may not have enough time to cover every aspect of vim (because it will take hours if not weeks/months to have a proper working setup), at the very least, I will get you to agree on just how much faster of a workflow it really is compared to most text-editors/IDE's out there.

Resources

Below are some of the resources I used to create the workflow that I currently have are:

- ▶ [Ej Mastnak: A Guide To Supercharged Mathematical Typesetting](#)
- ▶ [Gilles Castel: How I'm Able To Take Notes In Mathematics Lectures Using Latex and Vim](#)
- ▶ [The Primagean: 0 to LSP: Neovim From Scratch](#)

What is L^AT_EX?

- ▶ It is a high-quality typesetting system used for the production of technical and scientific documentation.
- ▶ It is the de-facto standard for communication and publication of scientific and scholarly articles.

What is Vim?

- ▶ Vim is an open sourced, terminal based modal text editor that is keyboard based, light-weight, and highly customizable.
- ▶ It is primarily used by software engineers to produce code without the use of a mouse!
- ▶ But this doesn't mean that software engineers are the only people that use vim.



What Are Some Ways That People Use Vim?

Other than programmers using it to write code, here are some ways that people might use vim:

- ▶ Write scripts for movies.
- ▶ Journal or creative writing
- ▶ Take notes during medical school

For us, we will primarily concern ourselves with writing math lecture notes.

Steep Learning Curve

Note that a lot of what you are about to see is a result of a large amount of time spent not only in setting up the editor, but also learning and being fluent with the key-binds and features of Vim. It cannot be emphasized enough that it takes A LOT of time to get used to an editor that is keyboard-centered. So, be patient when you are first setting up the editor and, expect and get used to the frustration when setting up your workflow. This effort will pay off tremendously in the long run.

What Do You Mean by Modal?

- ▶ Vim operates in different modes where each keystroke (or combination of keystrokes) executes different functions based on the current mode that you are in.
- ▶ There are three modes:
 - ▶ **Normal Mode (Command Mode):** This is the default mode of Vim. Keystrokes in this mode are meant for navigating around a file, manipulate text, and execute certain actions on text. Using the keys `h j k l` navigate the cursor around a file.
 - ▶ **Insert Mode:** This is the mode where you type in text into your file just as if you were typing on any other text-editor. To enable insert mode, you just have to press `i`.
 - ▶ **Command-Line Mode:** This mode allows you to execute more complex commands like saving files, quitting vim, and running external programs. To enter the command-line mode, `:`.

Examples of Common Commands

- ▶ Normal mode:
 - ▶ `cc` deletes a line and puts you into insert mode.
 - ▶ `di(` deletes any text that is wrap around in parentheses.
 - ▶ `ci[` deletes any text inside a set of brackets and puts you into insert mode.
 - ▶ `gg` takes to the top of the file
 - ▶ `G` takes you to the bottom of the file
- ▶ Command-line Mode:
 - ▶ `:w` writes the file (save)
 - ▶ `:q` quit vim
 - ▶ `:wq` writes the file and quits out of vim.

Why Do I Care?

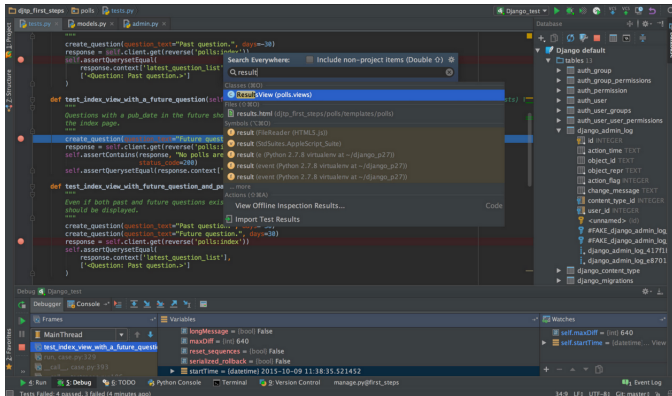
Vim being entirely keyboard based is perfect for optimizing the workflow of typesetting latex documents.

Problems Commonly Found in Modern Text-editors

- ▶ In most text-editors or IDE's, navigation revolves around the mouse which can be extremely inefficient.
- ▶ In IDE's, there are lots of potential for distraction due to the abundance of features available to you which may be overwhelming.
- ▶ Most programs have a large amount of "bloat" that will slow your program when latex files get large.

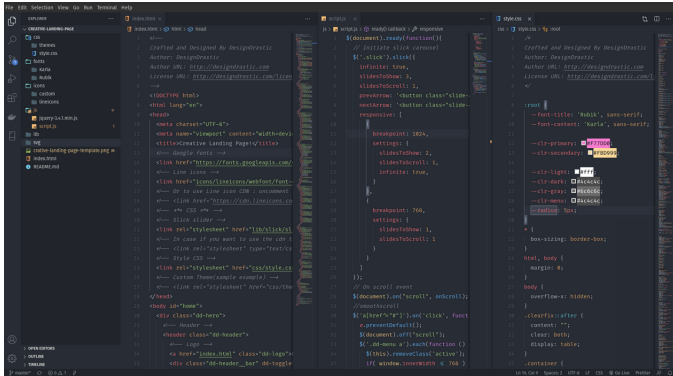
Example

Pycharm



Examples (Cont.)

VS Code (Visual Studio Code)



Problems Commonly Found Modern Text-Editors (Cont.)

- ▶ If you are using an online-based text editor, like overleaf, server crashes may halt your work and may even get rid of it altogether.
- ▶ Having your work stored locally in your machine helps prevents any potential of losing your work, losing time and gaining frustration that may come from it.



What Are The Advantages?

- ▶ Vim is lightweight and does not require any sophisticated hardware to be able to run properly. It doesn't even require the internet!
- ▶ There is no "GUI" needed to access the editor. You just need to use the terminal.
- ▶ It is entirely local which may ease the fear of having your files tampered with if you are using an online-based editor.

What Are The Advantages (Cont.)

- ▶ It is barebones by design which means it is not distracting when you open it up initially. However, do not confuse this with not having a lot features! It does have a lot of it, you just have to uncover it yourself.
- ▶ Since it is entirely keyboard based, you do not have to lift your hand off the keyboard to navigate a file which is HUGE.

The Most Annoying Aspect of $\text{\LaTeX}$

- ▶ With $\text{\LaTeX}$, all of the code produced involve typing " $\backslash$ " which is cumbersome to type out every single time.
- ▶ We can easily avoid this nuisance by creating "snippets" of all our commonly typed text.
- ▶ These snippets along with the keyboard centric workflow of vim makes for an extremely fast and efficient workflow.

What Is A Snippet?

- ▶ A snippet is a short reusable piece of text or code that can be triggered by entering a combination of keystrokes.
- ▶ These pieces of text are stored in a separate file that is executed every time you hit keystrokes corresponding to the text that you want expanded out.
- ▶ These snippets are triggered either manually or automatically.

Examples of Commonly Typed Text

- ▶ For inline math, type `\( \)`.
- ▶ Fractions, type `\frac{}{}`
- ▶ Kind of environments like `\begin{...} ... \end{...}`

Difference Between Autocomplete and Snippets

For those who are familiar with text-editors/IDE's, there may be some confusion between **autocomplete** and **snippets**.

- ▶ **Autocomplete** is a feature of most IDE's/text-editors that predicts what you want to type, presents options to you in some kind of a dropdown menu, and inserts the desired text that you have selected.
- ▶ In contrast, a **snippet** is already a written chunk of code or text that gets expanded based on certain action that triggers said text or code.

Just How Much Faster Is Vim At Notetaking Compared to Most Editors?

Answer: Much much faster. Blazingly fast.

Demo

Setup

That's great in all, but when are you going to show us how to set up these snippets?

What You Need To Setup Vim and L^AT_EX

- ▶ A working computer with either Windows, Mac OS, or Linux. (Out of all three of these, Mac OS is by far the easiest to setup and Windows being the hardest).
- ▶ Vim or Neovim (Obviously)
- ▶ Some knowledge on how to navigate directories in a terminal and install dependencies.
- ▶ The following dependencies are:
 - ▶ **A pdf viewer:** There are plenty of them out there. The one I use is called [Zathura](#).
 - ▶ **tex-live:** Contains the packages necessary for basic features to function and compile documents.
 - ▶ **Git:** For being able to download plugins created by other software developers to use in your setup. One of these being the main plugin called **vimtex**.
- ▶ Time, Patience, and the desire to tinker.

What You Need To Setup Vim and L^AT_EX(Cont.)

Once you have Neovim installed, you will need to set up a config file and write some code to configure Neovim to write latex files. You can set up keybindings, settings, and plugins all in one file if you want. However, if you want to have a more organized structure to your Neovim setup, I recommend to set it up in the following way:

What You Need To Setup Vim and L^AT_EX(Cont.)

```
/.config/nvim/  
├─ init.lua  
├─ lua/  
│   ├── packer.lua  
│   ├── plugins/  
│   │   ├── colorscheme.lua  
│   │   ├── lsp.lua  
│   │   ├── telescope.lua  
│   │   └─ ... (other plugin configurations)  
│   ├── settings.lua  
│   ├── keymaps.lua  
│   └─ ... (other Lua modules for configuration)  
└─ after/  
    └─ plugin/  
        └─ ... (optional, for overriding plugin  
              settings or loading after plugins)
```

Main Plugins to Install

- ▶ [Vimtex](#) (Filetype and syntax plugin for Neovim/Vim)
- ▶ [Luasnip](#) (Snippet Engine)
- ▶ [Telescope](#) (Filebrowser)

Optional:

- ▶ Language Server Protocol: [LSP](#) (Needed for code completion)
- ▶ [Tree-sitter](#) (Syntax Highlighter)

Snippets

Assuming that you have already you have your neovim setup configured and ready to go, it is now time to setup your snippets!

Location of Snippets

It is usually stored in a directory called "LuaSnip" and within this directory, you will create a file named `tex.lua` where you will store all of your snippets.

Anatomy of Snippet

```
6   ),
5   {
4       condition = in_mathzone
3   }
2 ),
1
182 s(
1   {
2       trig = 'int',
3       snippetType = 'autosnippet'
4   },
5   fmta("\\int_{<>}^{<>} <> \\ d<>",
6       {i(1), i(2), i(3), i(4)}),
7   {
8       condition = in_mathzone
9   }
10 ),
11
12 s(
13   {
14       trig = 'fint',
15       snippetType = 'autosnippet'
16   },
17   fmta(
18       [[\\int_{<>}^{<>} <> \\ d<> = <> - <>]],
19       {i(1), i(2), i(3), i(4), i(5), i(6)}
20   ),
main -- NORMAL -- tex.lua
"tex.lua" 1425L, 19798B written lua 12% 182:2
```

Structure of Snippet (Cont.)

- ▶ The variable `trig` represents the keystrokes needed to trigger the snippet.
- ▶ `snippetType` specifies whether your snippet will expand either automatically or manually.
- ▶ `[[ ]]` Inside these set of brackets will contain the text that, when the corresponding keystroke is applied, will be expanded out onto the screen.
- ▶ `{ }` will contain the nodes for the editor to jump to and from in order for the user to type in text needed to fill the empty space that the cursor occupies.
- ▶ `Condition` specifies whether the snippet will expand in math mode or not.

Code for Expanding Snippet

```
13     },
12   },
11 },
10 }
9
8 vim.keymap.set('i', "kk", function()
7   if ls.expand_or_jumpable() then
6     ls.expand_or_jump()
5   end
4     end, {silent = true})
3
2 -- Jumping backwards
1 vim.keymap.set('i', 'jj', function()
27 | if ls.jumpable(-1) then
1   ls.jump(-1)
2   end
3   end, {noremap = true, silent = true})
4 -- listing choice of snippets to choose from
5
6 vim.keymap.set('i', '<c-l>', function()
7   if ls.choice_active() then
8     ls.change_choice(1)
9   end
10 end)
11
12 vim.keymap.set('n',
13   '<Leader><leader>s',
| main -- NORMAL --
```

luansippet.lua

lua 76% 27:1

How I Use Snippets

My main purpose for using snippets is to be able to write lecture notes as my professors are writing on the board. We want a set up that is extremely fast (even faster than writing on paper) at doing this. The best practice for this is to set most of your snippets as auto expandable and set the keystroke to something that is optimized for your fingers to press conveniently fast. This is done by setting the `snippetType` variable to `"autosnippet"`.

Final Remarks

- ▶ As you progress through your mastery of this editor and setting up your workflow to fit your needs, you might be tempted to get sucked in to this mentality of needing to "optimize" your workflow as much as you can. While this is a worthwhile pursuit, it is important to realize that sometimes having a functional set up that is not necessarily perfect that gets work done is more valuable than spending all of your time tinkering to get that "perfect" setup and getting nothing done.
- ▶ All in all, if you are into tinkering, then vim is the perfect place for a time sink for you to explore in all of its grand features.